Credit Score Project - Simple Summary

Predict if a customer's credit score is Good, Standard or Poor

1. What is this project?

A bank has information about customers (income, loans, late payments, debt...). We teach a computer to look at this information and guess the customer's **credit score**: **Good**, **Standard** or **Poor**. The computer method we use is a **Decision Tree** - it works like a game of yes/no questions (for example: *"Is the credit mix good?" - yes - "Is the delay small?"* ...) until it reaches an answer.

2. The data

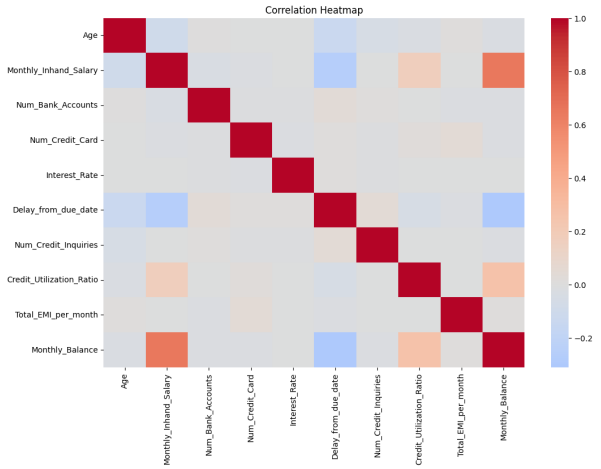
- Source: Kaggle, file **train.csv**. It has **3,380 rows** (customer records) and **28 columns** (details about each customer).
- The thing we predict is the column **Credit_Score**.
- The data was **messy**: numbers saved as text (like '35_'), junk symbols, missing values, and impossible ages (like -500).

3. How we cleaned the data

Problem	What we did
Useless columns (ID, Customer_ID, Name, SSN)	Removed them - they don't help predict
Junk symbols like '_' and '!@9#%8'	Cleaned them / treated as missing
Impossible ages (below 18 or above 100)	Marked as missing
1 row with no Credit_Score	Removed it (3,379 rows left)
Missing values	Filled numbers with the median, text with the most common value

4. Charts and what we learned





- **Not balanced:** Standard 53%, Poor 28%, Good 19%. So accuracy alone can fool us - we trust **Macro F1** more.
- People with higher salary tend to have higher monthly balance.
- People who pay late tend to have lower salary and balance.

5. Preparing for the model

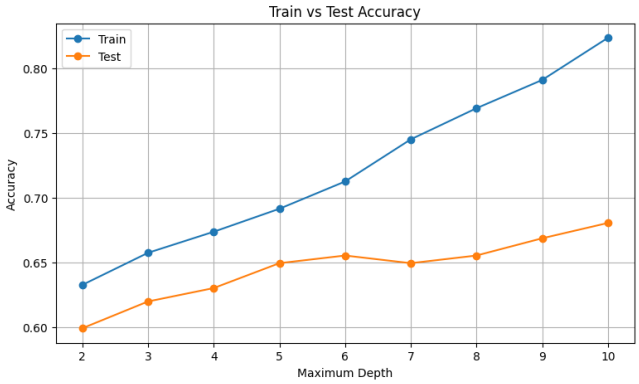
- **New feature we created:** Debt-to-Income = Outstanding Debt / Annual Income. It shows how big the debt is compared with income.
- Text columns were turned into 0/1 columns (one-hot encoding).
- Picked the **15 best columns** (SelectKBest).
- Split data: **80% to train (2,703 rows)** and **20% to test (676 rows)**.

6. We tried 12 different settings

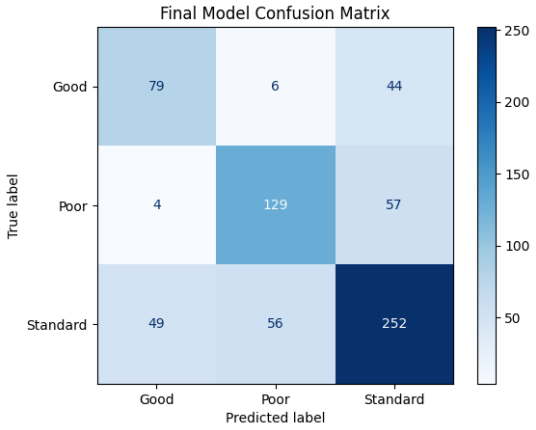
We tried the default tree, **gini** vs **entropy** (two ways to choose questions), and tree depth from 2 to 10. A few results:

Setting	Test accuracy	Macro F1
entropy (BEST)	68.05%	0.6641
max_depth = 10	68.05%	0.6597
gini / default	68.05%	0.6584
max_depth = 5	64.94%	0.6386
max_depth = 2 (worst)	59.91%	0.5935

- **Very shallow trees** (depth 2-4) are too simple, so they do badly.
- **Default tree** scores 100% on training data but only 68% on test data - it **memorised** the training data (this is called overfitting).
- **Best setting: entropy**, because it has the highest Macro F1.



7. Final result



Out of 676 test customers:
- **460 correct** (79 Good + 129 Poor + 252 Standard)
- **216 wrong**

Accuracy: 68%
Macro F1: 0.66
AUC: 0.74

Most mistakes are between neighbouring classes. Good-vs-Poor mix-ups are very rare (4 and 6).

Most important features: 1) Debt-to-Income ratio, 2) Monthly balance, 3) Delay from due date. The first question the tree asks is about **Credit_Mix_Good**.

8. Predicting 3 new customers

We took 3 customers from the test data and changed their salary by -10%, 0% and +10%. The model predicted **Standard** for all three. This is believable because Standard is the most common class, but it doesn't show the model separating classes - a small salary change isn't enough to change the answer.

9. Conclusion

- The model predicts the credit score correctly about **68%** of the time (Macro F1 = 0.66). That is better than always guessing Standard (53%), but only moderate.
- Debt compared with income, monthly balance and late payments matter most.
- We learned: real data needs a lot of cleaning; accuracy can mislead when classes are unequal; compare train vs test results to spot overfitting.